

2.28 Budget Calculator

Settings

Access controls

0 access controls enabled.

Beta features

LAB ACTIVITY

2.28.1: Budget Calculator

Lab: Personal Budget Simulator

Overview

You'll write a single Python script that models a simple personal budget: income, fixed expenses, goal, and a randomized "what if something unexpected happens" scenario.

Skills this lab is checking: `input()`, `print()`, output, data type conversions, and mathematical expressions

Required Sections

Your script must contain **Milestones 2-5**, in order, on one file.

Milestone 1 (20 pts): IDE setup

To get credit, you need to have an IDE installed.

Milestone 2 (20 pts): Income

1) Prompt for:

- **Hourly wage**, **hours worked per week**, and the **number of weeks** in the budgeting period.

2) Compute the **gross income** for the whole period from those three values.

- Gross income is calculated by multiplying the hourly wage, hours worked per week, and the number of weeks together.

3) Generate output to match the example below.

Milestone 3 (20 pts): Fixed Expenses

1) Prompt for three separate costs, each covering the **entire** budgeting period (not weekly or monthly): **rent**, a **utilities estimate**, and a **subscriptions total**.

2) Add them into a **running total**.

3) Generate output to match the example below.

Milestone 4 (20 pts): Variable Spending Estimate

1) Prompt for an estimated **daily** discretionary spending amount.

2) Compute **total discretionary spending** for the budgeting period. Think carefully about how to compute this. You already have the information you need from Milestone 1, so you should **not** ask the user for additional input.

3) Generate output to match the example below.

Milestone 5 (20 pts): Unexpected Expenses

1) Compute **leftover** money after fixed and discretionary expenses are subtracted from gross income.

2) Use input for 3 unexpected expenses:

- For each expense, use input to get the name and the cost of the expense.

3) Starting from your leftover amount (previously computed), subtract all three surprise expenses.

4) Compute the dollar difference between the original leftover and the adjusted leftover.

5) Generate output to match the example below.

Formatting requirements

- All dollar amounts should print with exactly two decimal places (e.g. `\$1234.50`, not `\$1234.5` c
- Include a clear banner/header so the output is easy to read top to bottom.

Deliverables

Submit all of the following together:

- 1) **One .py** file containing all five sections including comments and blank lines for spacing.
- 2) Your code should be commented. Not commenting your code will result in -10 points. All comments should be docstrings ("""This is a docstring""").
- 3) Each submission should have a description of what you changed for each submission. Not doing so will result in -10 points. Commenting your code.
- 4) To receive credit for the assignment, you need to complete a check off with one of the lab TAs
 - The TA will collapse all docstring comment blocks.
 - The TA will ask you questions about your code. If you cannot answer the TAs questions, you will lose points. Checkoffs can be attempted multiple times.

Example Output

Hourly wage: \$Hours worked per week: Number of weeks in this budgeting period:

=====

PERSONAL BUDGET SIMULATOR

=====

Gross income for 12 week(s): \$6000.00

Rent (the 12 week period): \$Utilities estimate (the 12 week period): \$Subscriptions total (the 12 w

Rent: \$1800.00

Utilities: \$450.00

Subscriptions: \$300.00

Total fixed: \$2550.00

Estimated daily discretionary spending: \$--- Discretionary Spending ---

Total discretionary spending: \$2100.00

That's 35.0% of gross income

Unexpected expense name: Unexpected expense cost: \$Unexpected expense name: Unexpected
Car Repair: \$300.0
Emergency Room: \$50.0
Birthday Gift: \$30.0
Original leftover: \$1350.00
Adjusted leftover: \$970.00
Difference: \$380.00

=====

Before you submit: self-check

- All five Milestones are present, in order, and clearly labeled.
- Every dollar amount prints with two decimal places.
- You can explain, out loud, why every variable in your file exists and where it gets used later.
- You ran the script at least twice with different inputs and verified the outputs.

[



▶ Run

main.py

1